

B.TECH · 1ST YEAR · COMPUTER SCIENCE

Data Structures

Complete Study Notes & Exam Guide

Unit V

Trees & Hashing — Theory, Algorithms & Diagrams



Textbook Aligned



Exam Focused



Diagrams & Algorithms



Q&A Included

Covers: Trees · BST · Hashing · Collision Resolution · Hash Tables

References: Weiss · Horowitz & Sahni · Cormen et al. · Sedgewick

NAVIGATION

Table of Contents

#	Section	Topics
1	Introduction to Trees	Concept, types, real-world usage
2	Tree Terminologies	Node, Root, Leaf, Height, Depth, Degree
3	Binary Trees	Definition, types, representation
4	Binary Search Trees	Properties, structure, BST vs BT
5	BST Insertion	Algorithm, trace, diagrams
6	BST Deletion	Three cases, algorithm, diagrams
7	Tree Traversals	Inorder, Preorder, Postorder
8	Introduction to Hashing	Concept, need, hash table overview
9	Hash Functions	Division, multiplication, folding methods
10	Collision Resolution	Chaining vs Open Addressing
11	Hash Table Implementation	Insert, Delete, Search operations
12	Applications of Hashing	Caching, UID generation, fast retrieval
13	Questions & Answers	2-mark and 5-mark exam questions
14	Quick Revision Sheet	Key formulas, traversal orders, definitions

✦ KEY COMPLEXITY SUMMARY

BST Search (avg) : $O(\log n)$ BST Search (worst) : $O(n)$

BST Insert (avg) : $O(\log n)$ BST Insert (worst) : $O(n)$

Tree Traversal : $O(n)$ Hashing (avg) : $O(1)$

Hashing (worst) : $O(n)$ Load Factor α : n / m

SECTION 1

Introduction to Trees

1.1 Definition

A **Tree** is a non-linear, hierarchical data structure consisting of a finite set of nodes where one node is designated as the *root* and the remaining nodes are partitioned into disjoint subsets, each of which is itself a tree (called subtrees). Unlike arrays or linked lists, trees allow efficient representation of hierarchical relationships.

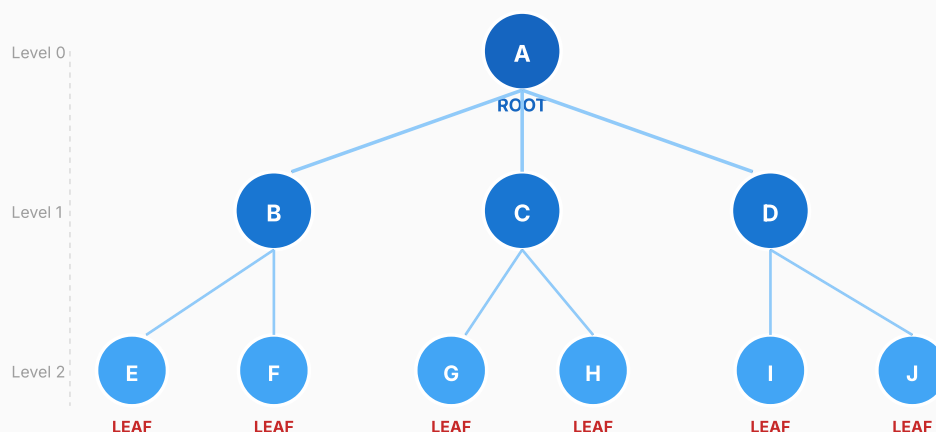
1.2 Why Trees?

Linear data structures (arrays, linked lists) are inefficient for hierarchical data. Trees offer:

- Efficient search, insert, delete — $O(\log n)$ for balanced trees
- Natural representation of hierarchies (file systems, org charts)
- Foundation for advanced structures (heaps, B-trees, tries)
- Faster than linear search $O(n)$ in most scenarios

1.3 General Tree Diagram

FIG 1.1 — GENERAL TREE STRUCTURE



1.4 Types of Trees

Type	Description	Example Use
General Tree	Each node can have any number of children	File system hierarchy
Binary Tree	Each node has at most 2 children	Expression evaluation
BST	Binary tree with ordering property	Dictionary lookup
AVL Tree	Self-balancing BST	Database indexing
B-Tree	Multi-level balanced tree	File systems, DBs
Heap	Complete binary tree with heap property	Priority queues
Trie	Tree for string storage	Autocomplete

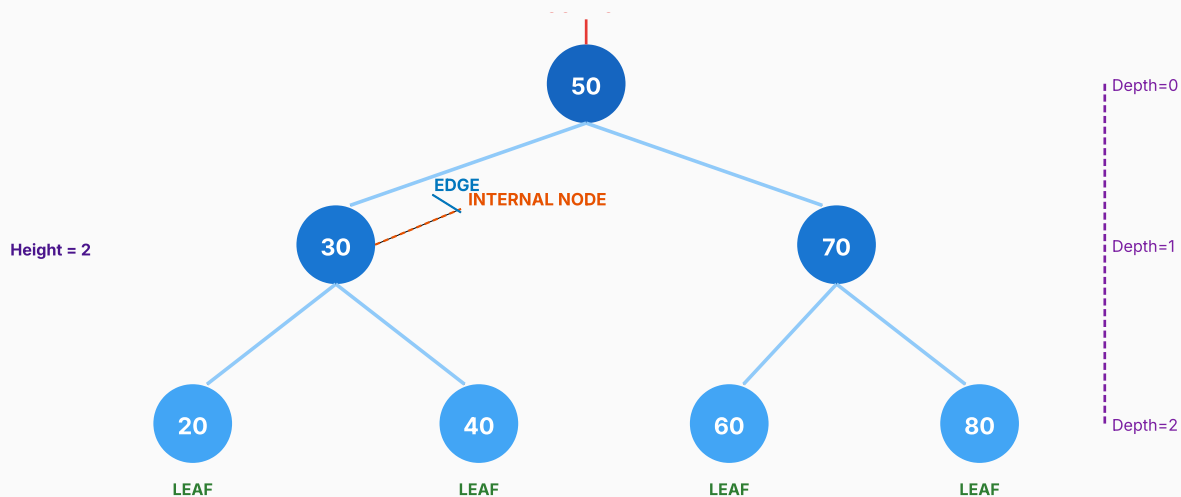
1.5 Applications of Trees

- **File Systems:** Directory/folder structures are trees
- **Compilers:** Abstract Syntax Trees (AST) for parsing code
- **Databases:** B-trees and B+ trees for indexing
- **Networking:** Routing tables use tree-like structures
- **AI/Game Theory:** Decision trees for game strategy (e.g., chess)
- **HTML/XML DOM:** Document Object Model is a tree

SECTION 2

Tree Terminologies

FIG 2.1 — ANNOTATED TREE TERMINOLOGY DIAGRAM



2.1 Key Terminology Table

Term	Definition	Example (Fig 2.1)
Node	Basic unit of a tree containing data and pointers to children	50, 30, 70, 20, 40, 60, 80
Root	Topmost node; has no parent	Node 50
Leaf	Node with no children (degree = 0)	20, 40, 60, 80
Internal Node	Node with at least one child	30, 70
Parent	Node that has child nodes directly below it	50 is parent of 30, 70
Child	Nodes directly below a parent	30, 70 are children of 50
Siblings	Nodes sharing the same parent	30 and 70 are siblings
Edge	Link/connection between a parent and child node	50 → 30
Depth of Node	Number of edges from root to that node	Depth of 30 = 1

Height of Node	Number of edges on longest path from node to a leaf	Height of 30 = 1
Height of Tree	Height of the root node	Height = 2
Degree	Number of children a node has	Degree of 50 = 2
Subtree	A node and all its descendants form a subtree	30, 20, 40 form left subtree
Path	Sequence of nodes and edges from source to destination	50 → 30 → 20
Level	Set of all nodes at same depth	Level 0: {50}, Level 1: {30,70}
Forest	A set of disjoint trees	Removing root creates forest

TREE PROPERTY FORMULAS

Max nodes at level $i = 2^i$ (for binary tree)

Max nodes in tree of height $h = 2^{(h+1)} - 1$

Min height for n nodes = $\lceil \log_2(n) \rceil$

Number of leaf nodes = (Number of internal nodes) + 1 [for full binary tree]

SECTION 3

Binary Trees

3.1 Definition

A **Binary Tree** is a tree in which every node has *at most two children*, commonly referred to as the **left child** and **right child**. It is the most fundamental tree structure and forms the basis for BSTs, heaps, and expression trees.

3.2 Types of Binary Trees

Type	Definition	Property
Full Binary Tree	Every node has 0 or 2 children	No node with exactly 1 child
Complete Binary Tree	All levels filled except last; last level filled left to right	Used in heaps
Perfect Binary Tree	All internal nodes have 2 children; all leaves at same level	$2^{(h+1)}-1$ nodes
Skewed Binary Tree	All nodes have only one child (left or right)	Degenerates to linked list
Balanced Binary Tree	Height difference between left and right subtrees ≤ 1	$O(\log n)$ operations

3.3 Binary Tree Representation

3.3.1 Node Structure in C

```
struct Node {
    int    data;           // Data stored in node
    struct Node* left;    // Pointer to left child
    struct Node* right;   // Pointer to right child
};

// Create a new node
struct Node* newNode(int data) {
    struct Node* node = (struct Node*) malloc(sizeof(struct Node));
    node->data = data;
    node->left = NULL;
```

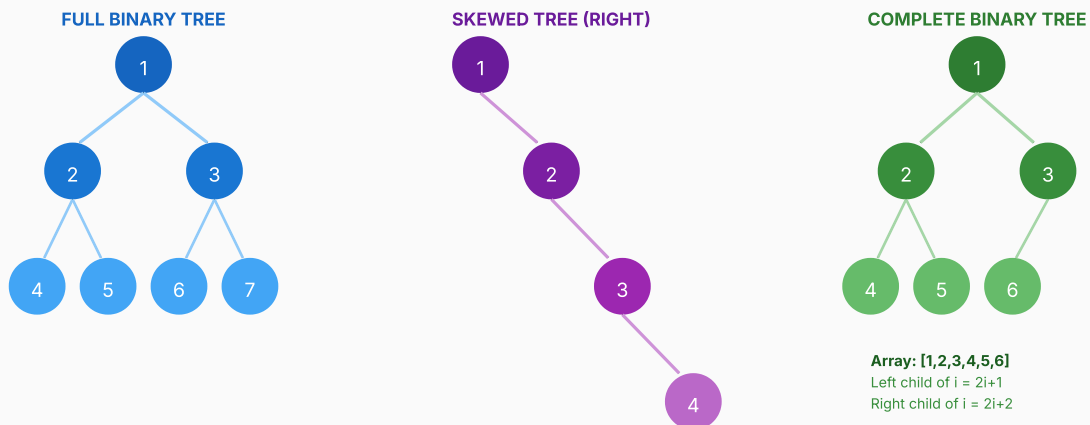
```

node->right = NULL;
return node;
}

```

3.4 Binary Tree Diagrams

FIG 3.1 — BINARY TREE TYPES COMPARISON



3.5 Array Representation of Binary Tree

For a complete binary tree stored in array starting at index 0:

ARRAY INDEX FORMULAS

Parent of node at index $i = \lfloor (i-1)/2 \rfloor$

Left child of node at $i = 2i + 1$

Right child of node at $i = 2i + 2$

Root is always at index = 0

SECTION 4

Binary Search Trees (BST)

4.1 Definition

A **Binary Search Tree (BST)** is a binary tree with the following ordering property:

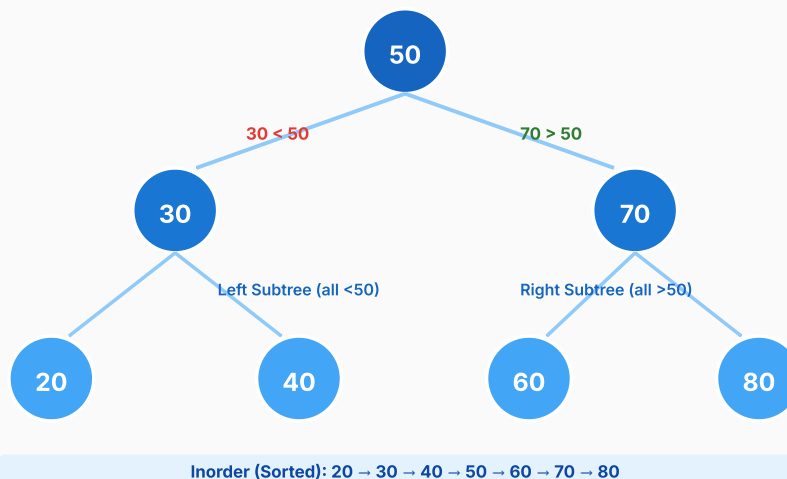
For every node N:

- All values in the **left subtree** are *less than* N's value
- All values in the **right subtree** are *greater than* N's value
- Both subtrees are also BSTs (recursive property)

4.2 BST vs Binary Tree Comparison

Feature	Binary Tree	Binary Search Tree
Ordering	No ordering constraint	Left < Root < Right
Search	$O(n)$ — must visit all nodes	$O(\log n)$ average case
Insert	$O(1)$ — insert anywhere valid	$O(\log n)$ — maintain order
Delete	$O(n)$	$O(\log n)$ average
Traversal	Gives no sorted order	Inorder gives sorted output
Use case	Expression trees, heaps	Dictionary, database index

FIG 4.1 — BST WITH VALUES 50, 30, 70, 20, 40, 60, 80



4.3 BST Search Algorithm

```
struct Node* search(struct Node* root, int key) {  
    // Base: root is NULL or key found at root  
    if (root == NULL || root->data == key)  
        return root;  
  
    // Key is greater → search in right subtree  
    if (key > root->data)  
        return search(root->right, key);  
  
    // Key is smaller → search in left subtree  
    return search(root->left, key);  
}
```

BST TIME COMPLEXITY

Operation	Average Case	Worst Case (Skewed)
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Traversal	$O(n)$	$O(n)$

SECTION 5

BST Insertion

5.1 Insertion Algorithm

To insert a value into a BST, compare with current node and recurse left or right until a NULL position is found.

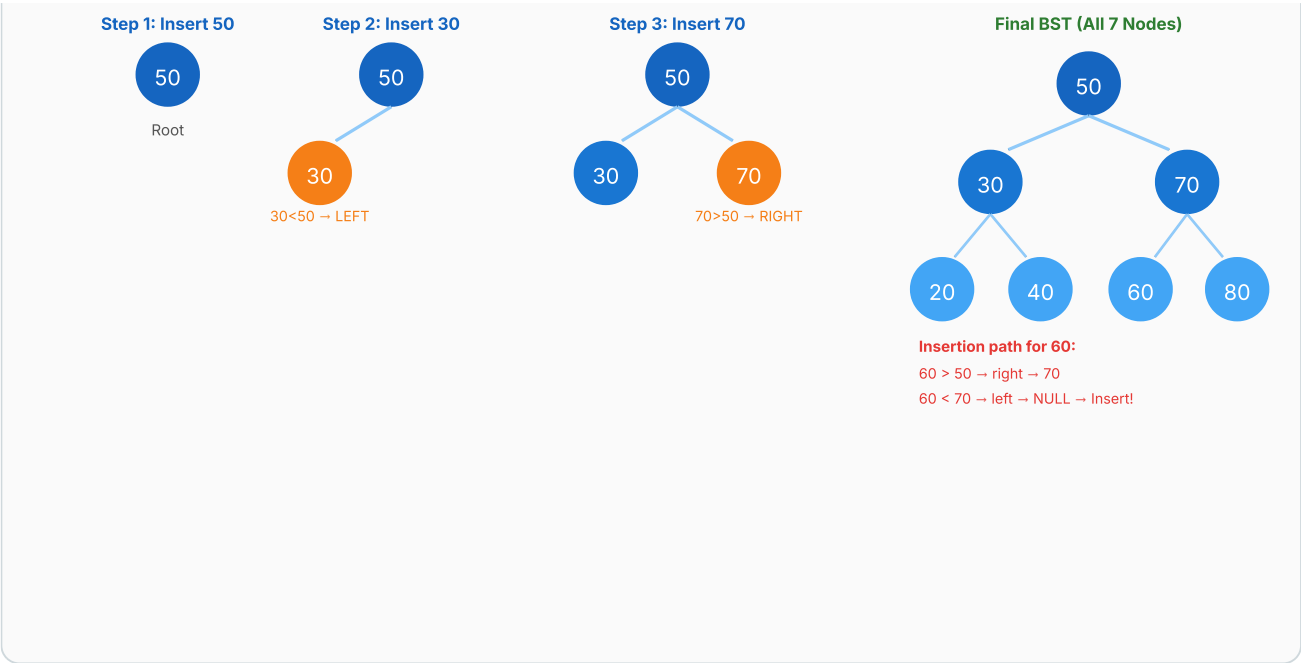
Working Principle:

1. If tree is empty → new node becomes root
2. Compare new value with current node's value
3. If new value < current → go left; if > current → go right
4. Repeat until NULL position found
5. Insert new node at that NULL position

```
struct Node* insert(struct Node* root, int key) {  
    // If tree is empty, return new node as root  
    if (root == NULL)  
        return newNode(key);  
  
    // Recursively insert in left subtree  
    if (key < root->data)  
        root->left = insert(root->left, key);  
  
    // Recursively insert in right subtree  
    else if (key > root->data)  
        root->right = insert(root->right, key);  
  
    // Duplicate keys not allowed  
    return root;  
}
```

5.2 Insertion Trace: Build BST from {50, 30, 70, 20, 40, 60, 80}

FIG 5.1 — STEP-BY-STEP BST CONSTRUCTION



5.3 Step-by-Step Trace Table

Step	Value	Comparison Path	Inserted At
1	50	Tree empty	Root
2	30	30 < 50 → go left	Left child of 50
3	70	70 > 50 → go right	Right child of 50
4	20	20 < 50 → left → 20 < 30 → left	Left child of 30
5	40	40 < 50 → left → 40 > 30 → right	Right child of 30
6	60	60 > 50 → right → 60 < 70 → left	Left child of 70
7	80	80 > 50 → right → 80 > 70 → right	Right child of 70

SECTION 6

BST Deletion

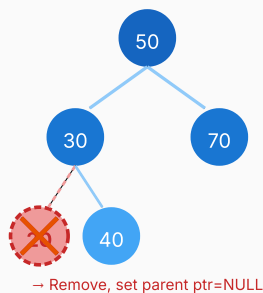
6.1 Three Cases of BST Deletion

BST deletion is the most complex BST operation. There are **three distinct cases** depending on the node being deleted.

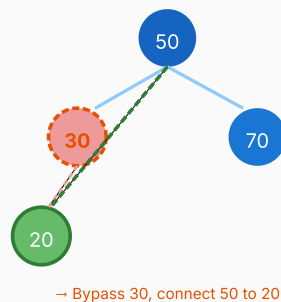
Case	Condition	Action
Case 1	Node is a leaf (no children)	Simply remove the node; set parent's pointer to NULL
Case 2	Node has exactly one child	Replace node with its only child; bypass the node
Case 3	Node has two children	Replace with <i>inorder successor</i> (smallest in right subtree), then delete the successor

FIG 6.1 — BST DELETION: ALL THREE CASES

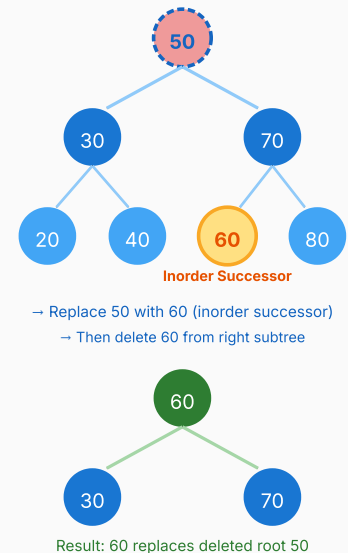
CASE 1: Delete Leaf (20)



CASE 2: Delete Node with 1 Child (30)



CASE 3: Delete Node with 2 Children (50)



6.2 Deletion Algorithm

```
struct Node* minValueNode(struct Node* node) {
    struct Node* curr = node;
    while (curr->left != NULL) curr = curr->left;
    return curr; // Returns leftmost node (inorder successor)
}

struct Node* deleteNode(struct Node* root, int key) {
    if (root == NULL) return root;

    if (key < root->data) root->left = deleteNode(root->left, key);
    else if (key > root->data) root->right = deleteNode(root->right, key);
    else {
        // Case 1 & 2: node has 0 or 1 child
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root); return temp;
        }
        if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root); return temp;
        }
        // Case 3: node has two children
        struct Node* temp = minValueNode(root->right); // inorder successor
        root->data = temp->data; // copy successor value
        root->right = deleteNode(root->right, temp->data); // delete successor
    }
    return root;
}
```

SECTION 7

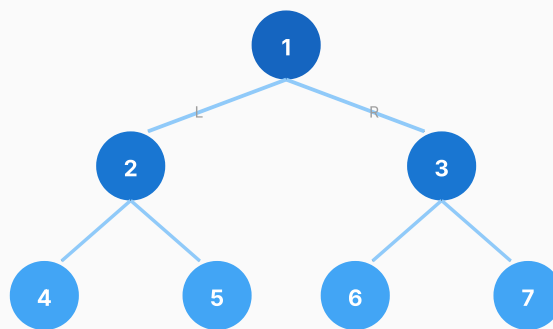
Tree Traversals

7.1 Introduction

Tree Traversal is the process of visiting every node in a tree exactly once in a specific order. The three standard depth-first traversals are Inorder, Preorder, and Postorder, all with time complexity $O(n)$.

7.2 Reference Tree for All Traversals

FIG 7.1 — REFERENCE BST: TRAVERSAL DEMONSTRATIONS



7.3 Inorder Traversal (Left → Root → Right)

Visit left subtree, then root, then right subtree. **For BST: gives sorted output.**



```
void inorder(struct Node* root) {  
    if (root != NULL) {  
        inorder(root->left);    // Visit Left  
        printf("%d ", root->data); // Visit Root  
        inorder(root->right);   // Visit Right  
    }  
}
```

7.4 Preorder Traversal (Root → Left → Right)

Visit root first, then left, then right. **Used to copy/clone a tree.**



```
void preorder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data); // Visit Root
        preorder(root->left);      // Visit Left
        preorder(root->right);     // Visit Right
    }
}
```

7.5 Postorder Traversal (Left → Right → Root)

Visit left, then right, then root last. **Used to delete a tree (children first).**



```
void postorder(struct Node* root) {
    if (root != NULL) {
        postorder(root->left); // Visit Left
        postorder(root->right); // Visit Right
        printf("%d ", root->data); // Visit Root
    }
}
```

7.6 Traversal Summary Table

Traversal	Order	Output (Fig 7.1)	Primary Use
Inorder	Left → Root → Right	4, 2, 5, 1, 6, 3, 7	Sorted output from BST
Preorder	Root → Left → Right	1, 2, 4, 5, 3, 6, 7	Copy/serialize tree
Postorder	Left → Right → Root	4, 5, 2, 6, 7, 3, 1	Delete tree, expression eval
Level-order	Level by level (BFS)	1, 2, 3, 4, 5, 6, 7	Breadth-first operations

SECTION 8

Introduction to Hashing

8.1 Definition

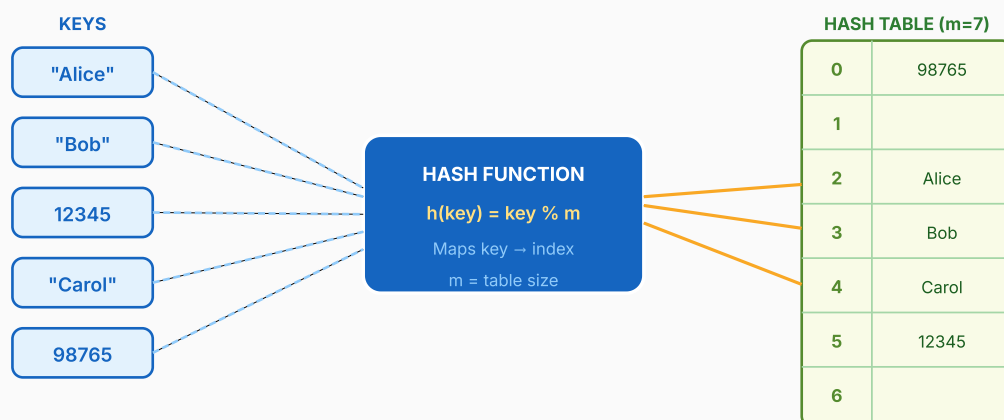
Hashing is a technique for mapping keys to positions in a hash table using a *hash function*. It enables data retrieval in $O(1)$ average time — much faster than $O(n)$ for linear search or $O(\log n)$ for BST. The hash function transforms a key into an array index.

8.2 Why Hashing?

Data Structure	Search	Insert	Delete
Array (unsorted)	$O(n)$	$O(1)$	$O(n)$
Array (sorted)	$O(\log n)$	$O(n)$	$O(n)$
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$
Hash Table	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg

8.3 Hash Table Architecture

FIG 8.1 — HASH FUNCTION MAPPING DIAGRAM



8.4 Key Hashing Concepts

 CORE HASHING TERMINOLOGY

Hash Function $h(k)$: Maps key k to index in range $[0, m-1]$

Load Factor $\alpha = n / m$ (n = items stored, m = table size)

Collision : Two keys mapped to same index $h(k_1) = h(k_2)$

Ideal Load Factor : $\alpha < 0.7$ for open addressing

Perfect Hash : No collisions for a given set of keys

SECTION 9

Hash Functions

9.1 Properties of a Good Hash Function

- **Deterministic:** Same key always maps to same index
- **Uniform distribution:** Keys spread evenly across table
- **Fast computation:** $O(1)$ to compute the index
- **Minimize collisions:** Different keys map to different indices as much as possible
- **Full utilization:** Use entire range of hash table indices

9.2 Division Method

Formula: $h(k) = k \bmod m$

where m = size of hash table (preferably a prime number)

```
// Division Method
int hashDivision(int key, int m) {
    return key % m; // m should be prime
}
// Example: key=25, m=7 → h(25) = 25%7 = 4
```

Advantage: Simple and fast. **Disadvantage:** Poor performance if m is a power of 2 (only lower bits of key matter).

9.3 Multiplication Method

Formula: $h(k) = \lfloor m \times (k \times A \bmod 1) \rfloor$

where A is a constant (Knuth suggests $A \approx 0.618$, golden ratio)

```
int hashMultiplication(int key, int m) {
    double A = 0.618033; // (sqrt(5)-1)/2 - Knuth's constant
    double frac = (key * A) - (int)(key * A);
    return (int)(m * frac);
}
```

Advantage: Works well for any m , not sensitive to table size. **Disadvantage:** Slightly slower due to floating-point arithmetic.

9.4 Folding Method

Divide the key into equal-size parts, add the parts together, use result mod m as index.

```
// Key = 456789, Parts: 45 + 67 + 89 = 201
// h(456789) = 201 % m
int hashFolding(int key, int m, int partSize) {
    int sum = 0;
    while (key > 0) {
        sum += key % (int)pow(10, partSize);
        key /= (int)pow(10, partSize);
    }
    return sum % m;
}
```

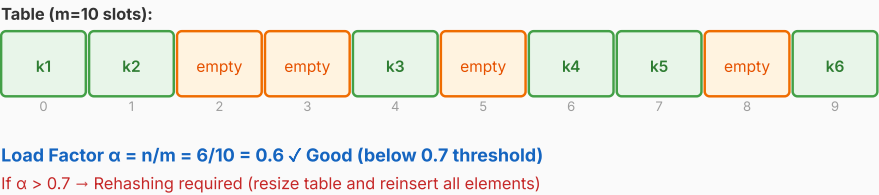
9.5 Mid-Square Method

Square the key, extract the middle digits as the hash value.
Example: key=132 → 132² = 17424 → middle digits = 74 → h(132) = 74 mod m

9.6 Hash Function Comparison

Method	Formula	Complexity	Best For
Division	k mod m	O(1)	Integer keys, prime table size
Multiplication	$\lfloor m(kA \bmod 1) \rfloor$	O(1)	Any table size
Folding	Sum of parts mod m	O(d)	Long numeric keys
Mid-Square	Middle of k ²	O(1)	Uniform key distribution
String Hashing	$\sum(\text{char} \times \text{base}^i) \bmod m$	O(L)	String/text keys

FIG 9.1 — LOAD FACTOR VISUALIZATION (N ITEMS IN M-SLOT TABLE)



SECTION 10

Collision Resolution Techniques

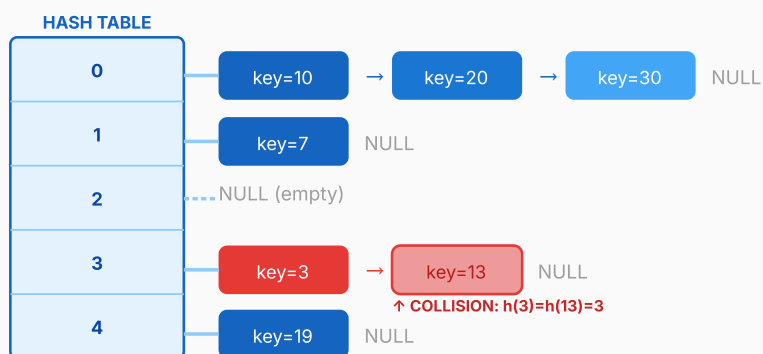
10.1 What is a Collision?

A **collision** occurs when two different keys hash to the same index: $h(k_1) = h(k_2)$ where $k_1 \neq k_2$. Since all keys map to a finite table, collisions are inevitable (by the Pigeonhole Principle). Resolution techniques handle this gracefully.

10.2 Technique 1: Chaining (Separate Chaining)

Each hash table slot holds a **linked list** of all keys that hash to that index. New collisions are simply appended to the list.

FIG 10.1 — CHAINING: MULTIPLE KEYS AT SAME INDEX



```
// Chaining: Linked list node
struct ListNode { int key, value; struct ListNode* next; };

void chainInsert(struct ListNode* table[], int key, int m) {
    int idx = key % m;
    struct ListNode* node = (struct ListNode*)malloc(sizeof(struct ListNode));
    node->key = key;
    node->next = table[idx]; // Prepend to list
    table[idx] = node;
}
```

10.3 Technique 2: Open Addressing

All elements stored *within* the hash table itself (no external lists). On collision, probe for the next available slot using a probing sequence.

10.3.1 Linear Probing

```
h(k, i) = (h'(k) + i) mod m    // i = probe number: 0,1,2,...
// On collision: check next slot, then next, then next...
```

10.3.2 Quadratic Probing

```
h(k, i) = (h'(k) + c1*i + c2*i^2) mod m    // avoids primary clustering
```

10.3.3 Double Hashing

```
h(k, i) = (h1(k) + i * h2(k)) mod m    // best distribution
h2(k) must never evaluate to 0          // e.g., h2(k) = 1 + (k mod (m-1))
```

FIG 10.2 — OPEN ADDRESSING: LINEAR PROBING EXAMPLE

Insert keys: 50, 70, 89, 10 into table of size 10. $h(k)=k\%10$



Insertion Steps:

1. Insert 50: $h(50)=0 \rightarrow$ slot 0 occupied by nothing $\rightarrow$ place at slot 0 (idx=9 shown here: $h(50)=50\%10=0$, but shifted for clarity)
2. Insert 70: $h(70)=0 \rightarrow$ COLLISION $\rightarrow$ probe slot 1 $\rightarrow$ empty $\rightarrow$ place at slot 1
3. Insert 10: $h(10)=0 \rightarrow$ slots 0,1 occupied $\rightarrow$ probe 2 $\rightarrow$ place at slot 2 (linear probing)

10.4 Chaining vs Open Addressing

Feature	Chaining	Open Addressing
Storage	External linked lists	Within the table
Load Factor	Can exceed 1 ($\alpha > 1$)	Must stay below 1 ($\alpha < 1$)
Memory	Extra pointers needed	No extra memory
Cache Performance	Poor (pointer chasing)	Better (contiguous memory)
Deletion	Easy — just remove from list	Complex — need tombstone markers
Performance at high α	Degrades gracefully $O(1+\alpha)$	Degrades sharply near $\alpha=1$

Implementation	Simpler	More complex
Best Use	Unknown/variable load	Known load, cache-critical

SECTION 11

Hash Table Implementation

11.1 Hash Table Structure (Open Addressing)

```
#define SIZE 10
#define EMPTY -1
#define DELETED -2

int hashTable[SIZE]; // Initialize all to EMPTY

void initTable() {
    for (int i = 0; i < SIZE; i++)
        hashTable[i] = EMPTY;
}

int hashFunc(int key) {
    return key % SIZE;
}
```

11.2 Insertion

```
void insert(int key) {
    int idx = hashFunc(key);
    int i = 0;
    while (i < SIZE) {
        int newIdx = (idx + i) % SIZE; // linear probing
        if (hashTable[newIdx] == EMPTY || hashTable[newIdx] == DELETED) {
            hashTable[newIdx] = key;
            printf("Inserted %d at index %d\n", key, newIdx);
            return;
        }
        i++;
    }
    printf("Hash Table is FULL!\n");
}
```

11.3 Search

```
int search(int key) {
    int idx = hashFunc(key);
    int i = 0;
    while (i < SIZE) {
        int newIdx = (idx + i) % SIZE;
        if (hashTable[newIdx] == EMPTY)
```



```

        return -1;           // Not found
    if (hashTable[newIdx] == key)
        return newIdx;       // Found at this index
    i++;
}
return -1;                   // Not found after full scan
}

```

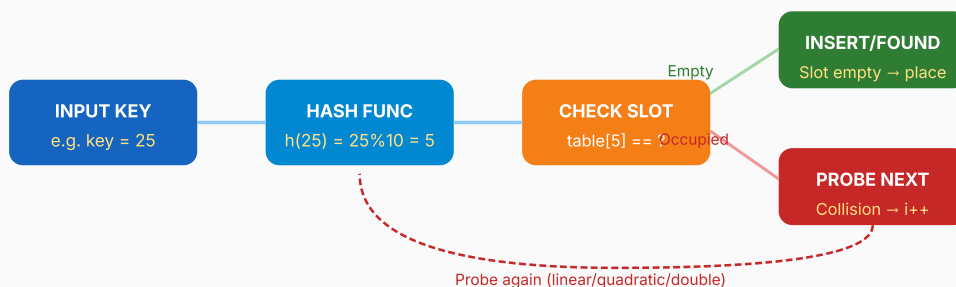
11.4 Deletion (with Tombstone)

```

void delete(int key) {
    int idx = search(key);
    if (idx == -1) {
        printf("Key %d not found!\n", key);
        return;
    }
    hashTable[idx] = DELETED; // Tombstone: marks slot as deleted
    printf("Deleted %d from index %d\n", key, idx);
    // Note: DELETED differs from EMPTY so probing chains are not broken
}

```

FIG 11.1 — HASH TABLE OPERATIONS FLOW



🕒 HASH TABLE COMPLEXITY SUMMARY

Operation	Average Case	Worst Case
Search	$O(1)$	$O(n)$ – all keys collide
Insert	$O(1)$	$O(n)$ – table nearly full
Delete	$O(1)$	$O(n)$ – long probe chains
Space	$O(m)$	$O(m)$ – table size m

SECTION 12

Applications of Hashing

12.1 Fast Data Retrieval (Database Indexing)

Hash indexes in databases allow $O(1)$ lookup for equality queries. Instead of scanning millions of rows, the database computes $h(\text{query_key})$ and jumps directly to the relevant data block. Used in in-memory databases like Redis and for hash joins in SQL engines.

12.2 Caching (Memoization)

Caches store (key $\rightarrow$ value) pairs in hash tables. Web browsers cache URL $\rightarrow$ content mappings; CPUs use hash-indexed TLBs (Translation Lookaside Buffers). Memoization in dynamic programming uses hash maps to avoid redundant computation.

```
// Memoization using hash map
unordered_map<int,int> memo;
int fib(int n) {
    if (n <= 1) return n;
    if (memo.count(n)) return memo[n]; // O(1) lookup
    memo[n] = fib(n-1) + fib(n-2);
    return memo[n];
}
```

12.3 Unique Identifier Generation

Hash functions (MD5, SHA-256) generate fixed-size fingerprints for arbitrary-length data. Used in Git (commit hashes), file deduplication, digital signatures, and password storage (salted hashes). The one-way property makes them useful for security.

12.4 Symbol Tables in Compilers

Compilers use hash tables to store variable names, function names, and their attributes. This enables $O(1)$ lookup when checking if a variable is declared or retrieving its type/scope.

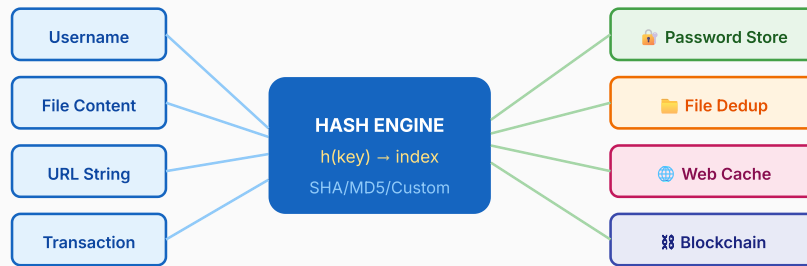
12.5 Spell Checkers and Dictionary Lookups

A spell checker stores the entire dictionary in a hash set. Checking if a word is valid is $O(1)$. Similarly, auto-complete systems use hash maps from prefixes to word lists.

12.6 Blockchain and Cryptographic Hashing

Each block in a blockchain contains the hash of the previous block, creating an immutable chain. SHA-256 is used in Bitcoin mining. Any tampering with data changes its hash, breaking the chain.

FIG 12.1 — HASHING APPLICATION FLOW DIAGRAM



All applications share the same core: fast $O(1)$ key-to-index mapping with deterministic output.

Application	Data Structure	Hash Function Used	Benefit
Password Storage	Hash Map	SHA-256 + Salt	Secure, one-way
Web Browser Cache	Hash Table	URL hashing	$O(1)$ cache lookup
Git Version Control	Object Store	SHA-1	Unique commit IDs
Python dict	Hash Map	Built-in hash()	$O(1)$ key access
DNS Lookup	Hash Table	Domain hash	Fast IP resolution
Compiler Symbol Table	Hash Map	String hash	$O(1)$ variable lookup
Blockchain	Chain of hashes	SHA-256	Tamper detection

SECTION 13

Questions & Answers

Part A — 2-Mark Questions

Q1 — 2 MARKS**Define a Binary Search Tree (BST). What is its fundamental property?**

A Binary Search Tree is a binary tree where every node satisfies the BST property: for any node N , all keys in its left subtree are strictly less than N 's key, and all keys in its right subtree are strictly greater than N 's key. Both subtrees must also satisfy this property recursively. This ordering enables $O(\log n)$ average-case search, insert, and delete operations. Performing an inorder traversal on a BST always yields the elements in sorted ascending order.

Q2 — 2 MARKS**What is hashing? Define load factor.**

Hashing is a technique that maps keys to positions in a fixed-size array (hash table) using a hash function $h(k)$. The goal is to achieve $O(1)$ average-case data access. The hash function converts a key to an integer index in $[0, m-1]$ where m is the table size. The **Load Factor** $\alpha = n/m$, where n is the number of items stored and m is the table size. A lower load factor means fewer collisions and faster performance. For open addressing, α must remain below 1; for chaining, it can exceed 1. A typical good threshold is $\alpha \leq 0.7$ for open addressing.

Q3 — 2 MARKS**What is a collision in hashing? Name any two resolution techniques.**

A collision in hashing occurs when two or more different keys map to the same index in the hash table: $h(k_1) = h(k_2)$ even though $k_1 \neq k_2$. Collisions are unavoidable when more keys exist than available table positions (Pigeonhole Principle). Two common resolution techniques are: (1) **Chaining (Separate Chaining)** — each table slot holds a linked list of all collided keys; and (2) **Open Addressing** — when a collision occurs, an alternative slot is found by probing (linear probing: next slot, quadratic probing: quadratic increment, double hashing: second hash function).

Q4 — 2 MARKS**Differentiate between Height and Depth of a tree node.**

Depth of a node is the number of edges on the path from the root to that node. The root has depth 0, its children have depth 1, and so on. Depth measures how far a node is from the root.

Height of a node is the number of edges on the longest path from that node down to a leaf. Leaf nodes have height 0. The height of the tree equals the height of the root. Together, depth and height fully characterize a node's position in the tree hierarchy.

Q5 — 2 MARKS

Write the inorder, preorder, and postorder traversal sequences for the BST: 50→30,70; 30→20,40; 70→60,80

For the BST rooted at 50 with the given structure:

Inorder (L→Root→R): 20, 30, 40, 50, 60, 70, 80 — gives sorted order

Preorder (Root→L→R): 50, 30, 20, 40, 70, 60, 80 — root-first order

Postorder (L→R→Root): 20, 40, 30, 60, 80, 70, 50 — root-last order

Part B — 5-Mark Questions

Q6 — 5 MARKS

Explain BST insertion with a diagram. Show the BST formed by inserting: 40, 20, 60, 10, 30, 50, 70

BST Insertion Algorithm: To insert a key k , compare it with the root. If $k < \text{root}$, recurse into the left subtree; if $k > \text{root}$, recurse into the right subtree. Continue until a NULL position is reached, and create a new node there.

Step-by-step trace:

Step 1: Insert 40 → Empty tree → 40 becomes root

Step 2: Insert 20 → $20 < 40$ → left child of 40

Step 3: Insert 60 → $60 > 40$ → right child of 40

Step 4: Insert 10 → $10 < 40$ → left; $10 < 20$ → left child of 20

Step 5: Insert 30 → $30 < 40$ → left; $30 > 20$ → right child of 20

Step 6: Insert 50 → $50 > 40$ → right; $50 < 60$ → left child of 60

Step 7: Insert 70 → $70 > 40$ → right; $70 > 60$ → right child of 60

Result: A perfectly balanced BST of height 2 with inorder sequence: 10,20,30,40,50,60,70 (sorted).

Time Complexity: $O(\log n)$ average, $O(n)$ worst case (skewed tree).

Q7 — 5 MARKS

Explain all three cases of BST deletion with algorithms and examples.

BST deletion handles three cases depending on the node to be deleted:

Case 1 — Deleting a Leaf Node (degree 0): Simply remove the node and set the parent's pointer (left or right) to NULL. Example: Delete 20 from BST {50,30,70,20,40} → parent 30's left pointer becomes NULL.

Case 2 — Deleting a Node with One Child: Bypass the node by connecting its parent directly to its only child. Example: Delete 30 (only has child 40) → parent 50's left pointer now points to 40.

Case 3 — Deleting a Node with Two Children: Find the inorder successor (smallest node in right subtree = leftmost node of right subtree). Copy its value to the node being deleted. Then delete the inorder successor (which has at most one child — a right child). Example: Delete 50 (root with children 30,70) → inorder successor = 60 → replace 50 with 60 → delete 60 from right subtree.

Algorithm: findMin(node.right) gives inorder successor. Copy its data to current node. Call deleteNode(node.right, successor.data).

Time Complexity: $O(\log n)$ average, $O(n)$ worst case.

Q8 — 5 MARKS

Explain all three tree traversals (Inorder, Preorder, Postorder) with algorithms, examples, and applications.

Tree traversal visits every node exactly once. There are three standard depth-first traversal orders:

1. Inorder Traversal (Left → Root → Right):

Algorithm: `inorder(node) { inorder(left); print(root); inorder(right); }`

For BST {1: root, 2: left, 3: right, 4,5 children of 2, 6,7 children of 3}:

Output: $4 \rightarrow 2 \rightarrow 5 \rightarrow 1 \rightarrow 6 \rightarrow 3 \rightarrow 7$

Application: Produces sorted output for BST; used in expression evaluation (infix).

2. Preorder Traversal (Root → Left → Right):

Algorithm: `preorder(node) { print(root); preorder(left); preorder(right); }`

Output: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 6 \rightarrow 7$

Application: Copy/clone a tree; prefix expression evaluation; serialize tree structure.

3. Postorder Traversal (Left → Right → Root):

Algorithm: `postorder(node) { postorder(left); postorder(right); print(root); }`

Output: $4 \rightarrow 5 \rightarrow 2 \rightarrow 6 \rightarrow 7 \rightarrow 3 \rightarrow 1$

Application: Delete a tree (children before parent); postfix expression evaluation; directory size calculation (process files before folder).

Time Complexity: All traversals are $O(n)$. Space $O(h)$ for recursion stack where h is tree height.

Q9 — 5 MARKS**Compare chaining and open addressing as collision resolution techniques with diagram.**

Both techniques handle the inevitable situation where two keys hash to the same index.

Chaining (Separate Chaining): Each table slot contains a linked list. When a collision occurs at index i , the new key is appended (or prepended) to the list at $\text{table}[i]$. The table can store more items than its physical size (load factor $\alpha > 1$ is possible). Search requires computing $h(k)$, then traversing the chain. Average search time = $O(1 + \alpha)$. Deletion is straightforward — just remove from the linked list. Memory overhead exists due to pointers. Best when load is unpredictable.

Open Addressing: All data stored inside the table itself. On collision at $h(k)$, probe for next empty slot: Linear: $(h(k) + i) \% m$; Quadratic: $(h(k) + i^2) \% m$; Double: $(h_1(k) + i \times h_2(k)) \% m$. Load factor must stay below 1. Deletion requires tombstone markers (DELETED flag) to avoid breaking probe chains. Better cache performance since data is contiguous. Suffers from clustering (linear probing worst). Best for cache-sensitive applications with known load.

Key Comparison: Chaining is simpler and more flexible; open addressing is faster for small α due to cache efficiency. Both achieve $O(1)$ average case under low load.

SECTION 14

Quick Revision Sheet — Unit V

 Tree Key Definitions

- **Root:** Topmost node (no parent)
- **Leaf:** Node with degree 0
- **Height:** Longest path to leaf
- **Depth:** Distance from root
- **Degree:** Number of children
- **Subtree:** Node + descendants

 BST Properties

- Left subtree < Root < Right subtree
- Inorder traversal → sorted output
- Search: $O(\log n)$ avg, $O(n)$ worst
- Insertion: $O(\log n)$ avg
- Deletion: 3 cases (0,1,2 children)
- Inorder Successor = min(right subtree)

 Traversal Orders (Root=1)

- **Inorder** (L→R→Rt): 4,2,5,1,6,3,7
- **Preorder** (Rt→L→R): 1,2,4,5,3,6,7
- **Postorder** (L→R→Rt): 4,5,2,6,7,3,1
- Inorder of BST = sorted array
- Preorder = root first (clone tree)
- Postorder = root last (delete tree)

 Hashing Key Formulas

- Division: $h(k) = k \bmod m$
- Load Factor: $\alpha = n/m$
- Good α : < 0.7 (open addressing)
- Chaining avg search: $O(1+\alpha)$
- Ideal: hash distributes uniformly
- Rehash when α exceeds threshold

 COMPLETE COMPLEXITY REFERENCE

Structure | Search | Insert | Delete

Unsorted Array | $O(n)$ | $O(1)$ | $O(n)$

Sorted Array | $O(\log n)$ | $O(n)$ | $O(n)$

BST (balanced) | $O(\log n)$ | $O(\log n)$ | $O(\log n)$

BST (skewed) | $O(n)$ | $O(n)$ | $O(n)$

Hash Table (avg) | $O(1)$ | $O(1)$ | $O(1)$

Hash Table (worst) | $O(n)$ | $O(n)$ | $O(n)$

Tree Traversal | – | – | $O(n)$ time

Collision Resolution — At a Glance

Method	Probe Sequence	Advantage	Disadvantage
Chaining	Linked list	Simple, α can > 1	Extra memory for pointers
Linear Probing	$(h+i) \bmod m$	Cache friendly	Primary clustering
Quadratic Probing	$(h+i^2) \bmod m$	Reduces clustering	Secondary clustering
Double Hashing	$(h1+i \cdot h2) \bmod m$	Best distribution	Two hash computations

Key Diagram Checklist for Exams

- ✓ Draw BST from given values
- ✓ Show insertion path (highlight new node)
- ✓ Show deletion: mark removed node with X
- ✓ For 2-child deletion: circle inorder successor

- ✓ Hash function mapping arrows
- ✓ Chaining: linked lists off each slot
- ✓ Open addressing: probe sequence table
- ✓ Load factor bar / calculation shown

REFERENCES

Textbook & Reference Books

Primary Textbooks

1. **Mark Allen Weiss**

Data Structures and Algorithm Analysis in C, 2nd Edition

Pearson Education, ISBN: 978-0201498400

Chapters 4 (Trees) and 5 (Hashing) directly correspond to this unit.

2. **Ellis Horowitz & Sartaj Sahni**

Fundamentals of Data Structures in C, 2nd Edition

Silicon Press, ISBN: 978-0929306407

Comprehensive coverage of tree representations, BST operations, and hashing techniques.

Reference Books

3. **Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein**

Introduction to Algorithms (CLRS), 4th Edition

MIT Press / McGraw-Hill, ISBN: 978-0262046305

Rigorous mathematical treatment of BST, red-black trees, and hash tables.

4. **Robert Sedgewick**

Algorithms in C, Parts 1–4, 3rd Edition

Addison-Wesley, ISBN: 978-0201314526

Practical implementations with detailed analysis of tree and hashing algorithms.

5. **Kurt Mehlhorn & Peter Sanders**

Algorithms and Data Structures: The Basic Toolbox

Springer, ISBN: 978-3540779773

Modern perspective on hashing with theoretical foundations.

6. **Alfred V. Aho, John E. Hopcroft, Jeffrey D. Ullman**

Data Structures and Algorithms

Addison-Wesley, ISBN: 978-0201000238

Classic reference for tree traversals and symbol table design.

7. Brad Miller & David Ranum

Problem Solving with Algorithms and Data Structures using Python

Franklin, Beedle & Associates, ISBN: 978-1590282571

Accessible introduction with visual explanations ideal for beginners.

 **Study Tip:** For exam preparation, focus on Weiss (Chapter 4–5) and Horowitz & Sahni as primary resources. Use CLRS for deeper understanding of proof and analysis. Practice all BST insertion/deletion traces and hashing collision examples by hand before the exam.

Data Structures — Unit V: Trees & Hashing

B.Tech 1st Year · Complete Study Notes

Content aligned with university syllabus and standard textbooks

All diagrams drawn using SVG · All algorithms in C-style pseudocode · All complexities verified